

Andre Reyes
Operating Systems: Lab 1
Due: 9/28/23

Objective

The objective of this assignment is to work with creating and executing multiple processes. This involves using the `fork()` function in order to create a copy, the child, of the parent process. Once created the child processes exit first before the parent is allowed to end, otherwise the child will become a “zombie.”

Part 2 expands on this concept by allowing an integer to be taken as an argument, `n`, and executes 2^n processes and then terminates.

Algorithms & Functions

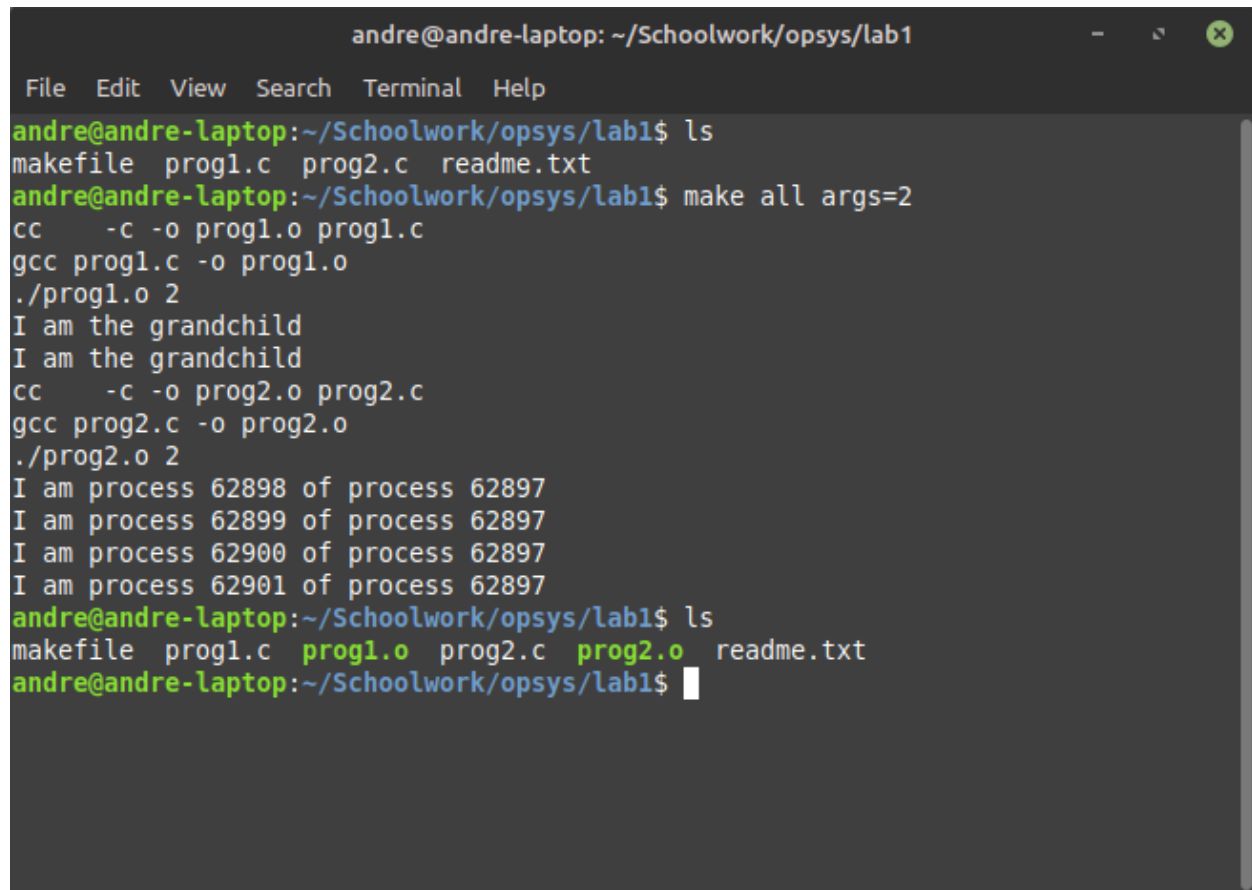
`fork()`: creates a child process from the original parent.

`waitpid()`: Allows the parent to wait for the child to exit before exiting itself.

`execve(“command location”, arguments array, argument pointer array)`: This function allows the process to overwrite it and execute a new process. In this case it was used to replace the child processes.

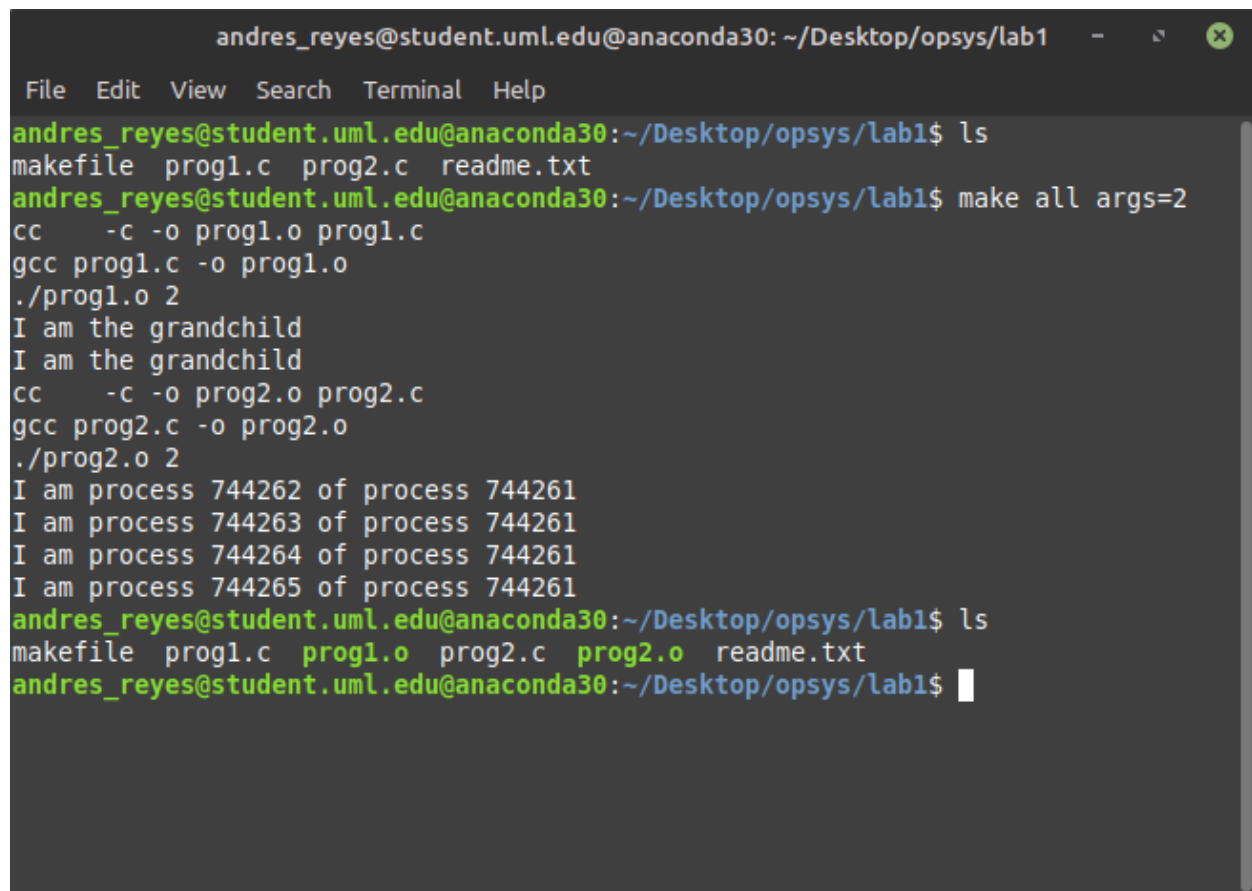
Results

Local:

A terminal window titled 'andre@andre-laptop: ~/Schoolwork/opsys/lab1' showing the execution of a program. The user runs 'ls' showing 'prog1.c', 'prog2.c', and 'readme.txt'. Then they run 'make all args=2', which compiles 'prog1.c' to 'prog1.o' and 'prog2.c' to 'prog2.o'. They then run './prog1.o 2', which prints 'I am the grandchild' twice. Finally, they run './prog2.o 2', which prints 'I am process 62898 of process 62897', 'I am process 62899 of process 62897', 'I am process 62900 of process 62897', and 'I am process 62901 of process 62897'. The terminal ends with another 'ls' command showing 'prog1.o' and 'prog2.o' in addition to the other files.

```
andre@andre-laptop: ~/Schoolwork/opsys/lab1
File Edit View Search Terminal Help
andre@andre-laptop:~/Schoolwork/opsys/lab1$ ls
makefile prog1.c prog2.c readme.txt
andre@andre-laptop:~/Schoolwork/opsys/lab1$ make all args=2
cc -c -o prog1.o prog1.c
gcc prog1.c -o prog1.o
./prog1.o 2
I am the grandchild
I am the grandchild
cc -c -o prog2.o prog2.c
gcc prog2.c -o prog2.o
./prog2.o 2
I am process 62898 of process 62897
I am process 62899 of process 62897
I am process 62900 of process 62897
I am process 62901 of process 62897
andre@andre-laptop:~/Schoolwork/opsys/lab1$ ls
makefile prog1.c prog1.o prog2.c prog2.o readme.txt
andre@andre-laptop:~/Schoolwork/opsys/lab1$
```

Remote:

A terminal window titled 'andres_reyes@student.uml.edu@anaconda30: ~/Desktop/opsys/lab1'. The window contains the following text:

```
andres_reyes@student.uml.edu@anaconda30:~/Desktop/opsys/lab1$ ls
makefile prog1.c prog2.c readme.txt
andres_reyes@student.uml.edu@anaconda30:~/Desktop/opsys/lab1$ make all args=2
cc -c -o prog1.o prog1.c
gcc prog1.c -o prog1.o
./prog1.o 2
I am the grandchild
I am the grandchild
cc -c -o prog2.o prog2.c
gcc prog2.c -o prog2.o
./prog2.o 2
I am process 744262 of process 744261
I am process 744263 of process 744261
I am process 744264 of process 744261
I am process 744265 of process 744261
andres_reyes@student.uml.edu@anaconda30:~/Desktop/opsys/lab1$ ls
makefile prog1.c prog1.o prog2.c prog2.o readme.txt
andres_reyes@student.uml.edu@anaconda30:~/Desktop/opsys/lab1$
```

Observations:

One of the interesting aspects of this lab was that the processes sometimes would create a new parent process and other times would be the same parent process with incrementing children processes. I wasn't sure which was more correct, But The children would always reference the same parent they originated from when printing before and after the child process had exited.

I did have issues with the `execve()` function as It wasn't clear to me how this functioned in terms of rewriting the current process. I didn't know that linux commands were stored in a locatable file that this function points to. After much troubleshooting I finally understood how the command portion of it worked and was able to use `echo` to display the desired text within the process. This was necessary since `execve()` does not complete any code after its call, unless there was an error.

Conclusion

The lab was straight forward, so most tasks were executed as intended the first time I tried. Learning the makefile syntax was interesting. At first I was inputting the commands manually every time, but that quickly got tedious and I understood why makefiles existed. Getting arguments to pass along and have the output run within the make command had a lot of troubleshooting. It was worth it in the end to be able to have one command that does it all and potentially be reusable.

Source Code

Prog1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char* argv[]){
    int counter = 0;
    pid_t pid;
    int prog_repeat = atoi(argv[1]);
    //checks if number of arguments matches as intended to avoid conflict
    if (argc != 2)
    {
        printf("Format must be: [file location] [repeat (int) times]\n");
        exit(0);
    }

    do{
        pid = fork();
        if (pid < 0){
            perror("fork");
            exit(0);
        }
        //This is the child block
        if (pid == 0){
            char *args[] = {"echo","I am the grandchild", NULL};
            char *argp[] = {NULL};
            int res = execve("/bin/echo", args, argp);

            if(res == -1){
                perror("execv");
                exit(2);
            }
        }
    }
```

```

}
//This is the parent block
else{

int child_pid = pid;

waitpid(child_pid, NULL, 0);
// printf("Child %d exited from %d\n", child_pid, getpid());
}
counter ++;

} while (counter < prog_repeat);
return 0;
}

```

Prog2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(int argc, char* argv[]){
int counter = 0;
pid_t pid;
int n = atoi(argv[1]);
if (argc != 2 || n < 1 || n > 4)
{
printf("Format must be: [file location] [int value 1-4]\n");
exit(0);
}
for (int i = 0; i <= (2*n - 1); i++)
{
pid = fork();

```

```
if (pid < 0){
perror("fork");
exit(0);
}
//This is the child block
if (pid == 0){
printf("I am process %d of process %d\n", getpid(), getppid());
exit(0);
}
//This is the parent block
else{
int child_pid = pid;
waitpid(child_pid, NULL, 0);
}
counter ++;

}
return 0;
}
```